

공세민

Se Min Kong

로봇과 소프트웨어를 연결하는 개발자

안녕하세요. 공세민입니다.

숭실대학교에서 소프트웨어를 전공했습니다.

현재 SSAFY Robotics Track에서 공부하고 있습니다.

센서 입력부터 제어 명령과 실제 동작까지 연결하며,
동작 조건과 실패 경계를 함께 살펴봅니다.

semin1224@gmail.com

github.com/SeMinKong



현재

SSAFY Robotics Track
2026.01 - 현재

학력

숭실대학교
소프트웨어학부
2020.03 - 2026.02

관심 분야

Computer Vision
Robotics / Physical AI

거주지

Suwon,
Republic of Korea

프로젝트

장비 연동과 실물 제어를 중심으로, 입력과 출력의 품질을 다룬 작업을 소개합니다.

쪽	프로젝트	역할	핵심 판단
3 - 6	THING	6인 팀 / 구동·기구 통합	기구 편차 / 초기 목표와 토크 순서
7 - 9	AQIS	2인 팀 / 팀장·서버·장비 통합	장비 연결 분리 / 좌표 갱신과 집기
10 - 11	Briefit	6인 팀 / AI 담당	입력 정제 / 요약과 정보 보존
12 - 13	Brain MRI	개인 / 전처리·학습·통합 추론	라벨 변환 / 분류·분할 결합
14 - 15	Prompt Generator	개인 / 대화 서버·상태 관리	영역별 이력 / 수정 흐름
16 - 17	Alkkagi.io	개인 / 클라이언트·서버·물리	충돌·검침 보정 / 서버 입력 제한

THING · 텐던 로봇 핸드

기간

2026.07 - 08

팀·역할

6인 팀

모터 제어·기구 통합

사용 도구

DYNAMIXEL / U2D2

모터 점검·제어 스크립트

팀 기술

ROS 2 / MediaPipe

OpenCV / Jetson

Raspberry Pi 5



통합 조립

프로젝트 개요

카메라로 읽은 사람의 손동작을 텐던 로봇손으로 재현합니다. 손동작 인식부터 ROS 2 제어, 모터 구동과 관제까지 연결한 팀 프로젝트입니다.

담당 작업

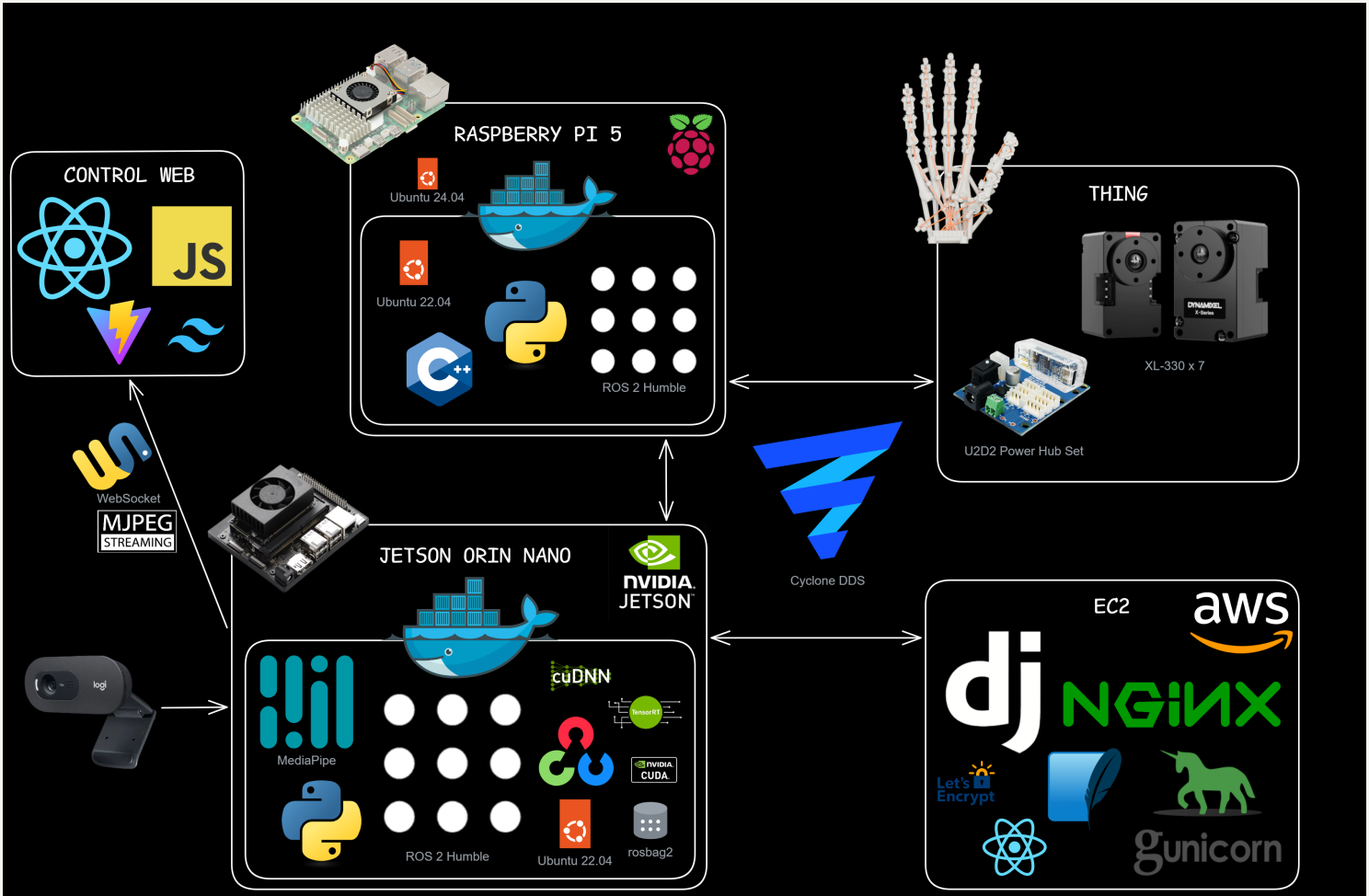
U2D2 통신 환경과 개별·원위치·정지 제어 스크립트를 작성하고, 아크릴 모터 고정부를 제작해 스프링·텐던과 조립했습니다. 소프트웨어의 위치 명령이 실제 손가락 동작으로 이어지는 구동부를 맡았습니다.

협업 범위

손동작 인식, ROS 2 명령 중재·guard, 관제와 데이터 기록은 팀의 협업 결과입니다. 본인 기여는 모터 제어와 기구 통합입니다.

THING · 시스템 구조

인식·제어권 중재·구동을 연결한 팀 구조와 독립 점검 경로



제어권을 거치는 운영 경로

모방·웹·키보드 입력을 선택한 뒤 guard를 거쳐 모터 명령을 전달합니다. 인식은 Jetson, 제어권 중재와 장치 제어는 Raspberry Pi가 담당합니다.

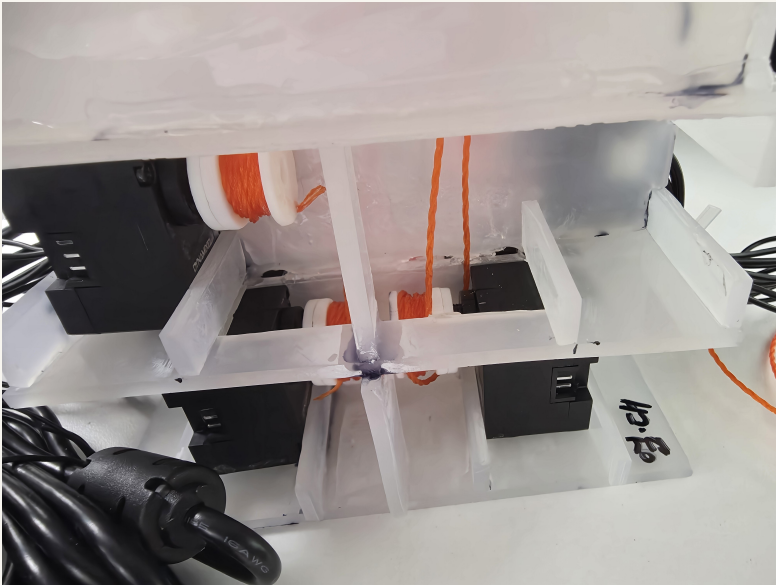
구동부를 따로 점검

본인 점검 스크립트는 통신 응답과 개별 구동을 확인하는 독립 도구입니다. 운영 경로의 명령 중재·guard와 구분해 조립 단계의 구동부를 점검했습니다.

원본: 최종 발표자료 25쪽. DDS는 내부 ROS 2 통신이며, Jetson → EC2 기록 전송은 HTTPS입니다.

THING · 기구 편차와 제어 조건

같은 모터 위치 명령도 텐던 장력과 권취 방향에 따라 실제 손가락 자세가 달라집니다.



구동부 내부: 모터·스풀·텐던 연결



전원부 모터 고정부

위치 명령과 실제 굽힘

통신 응답이 정상이어도 원하는 자세가 나오지는 별도 확인이 필요합니다. 모터 점검과 실제 기구 동작을 나눠 확인했습니다.

재장착 조건의 보존

모터 고정부를 제작하고 스푼·텐던을 통합했습니다. 체결 방향과 텐던 경로를 사진·작업일지에 남겨 재장착 시 참고할 기준을 마련했습니다.

이동 명령의 토크 순서

이동 명령에서는 현재 위치 읽기 → 목표값 기록 → 토크 ON 순서를 적용합니다. 이전 목표값으로 갑자기 이동하는 것을 막기 위한 점검 코드입니다.

원위치와 정지

다회전 위치에서 가장 가까운 중앙각 좌표를 선택합니다. 키보드로 개별·전체 모터의 토크를 끄는 Torque OFF 경로도 마련했습니다.

축별 보정의 경계

중앙각 복귀만으로 손가락의 가동 범위가 보정되지는 않습니다. 당시 일지에서 축별 끝점 보정과 최대 가동 범위의 간섭 검증을 후속 과제로 남겼습니다.

THING · 실물 동작과 결과



캔 파지 시연

수상

SSAFY 공통 프로젝트 우수상

2026.08.10 / THING

[상장 보기](#)

팀 시연

손동작 모방과 손가락 순차 동작, 캔·부드러운 물체 파지를 시연했습니다. 인식과 제어 명령이 조립한 구동부를 거쳐 실제 손가락 동작으로 이어졌습니다.

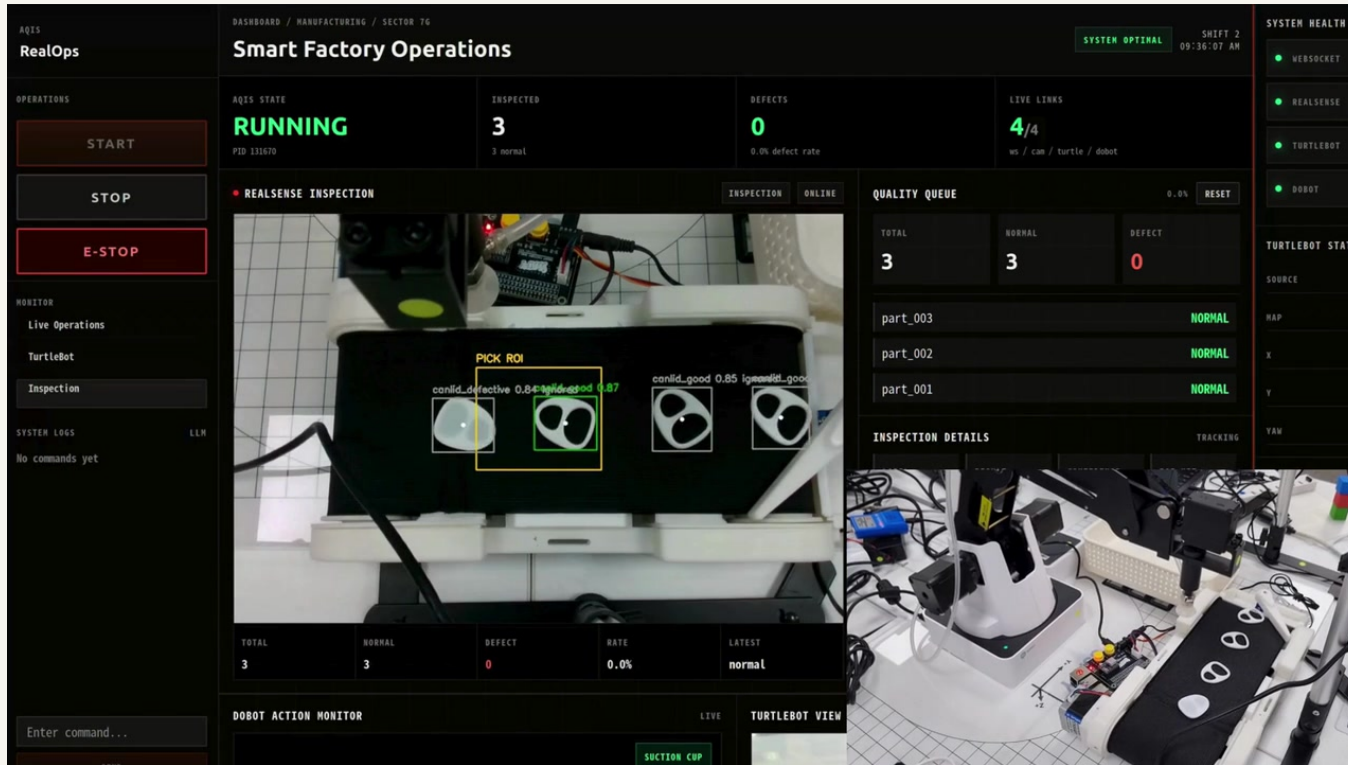
시연 이후의 판단

동작 시연은 시스템 연결의 근거입니다. 물체별 파지 성공률과 재조립 후 반복성을 판단하려면 장력·가동 범위와 시험 조건을 고정한 반복 측정이 필요합니다.

보정 전후 편차·반복 파지 성능의 정량 결과는 공개 기록에서 확인되지 않았습니다.

AQIS · 스마트 팩토리

검사, 컨베이어, Dobot과 RealOps 관제를 연결한 스마트 팩토리 시스템



RealOps · 실제 장비 시연

협업 팀원은 모델 학습·Roboflow·CAD·시뮬레이션을 담당했습니다.

2026.05 기획 / 06 본 개발

2인 팀 · 팀장

Full-stack & Robot Integration

프로젝트 개요

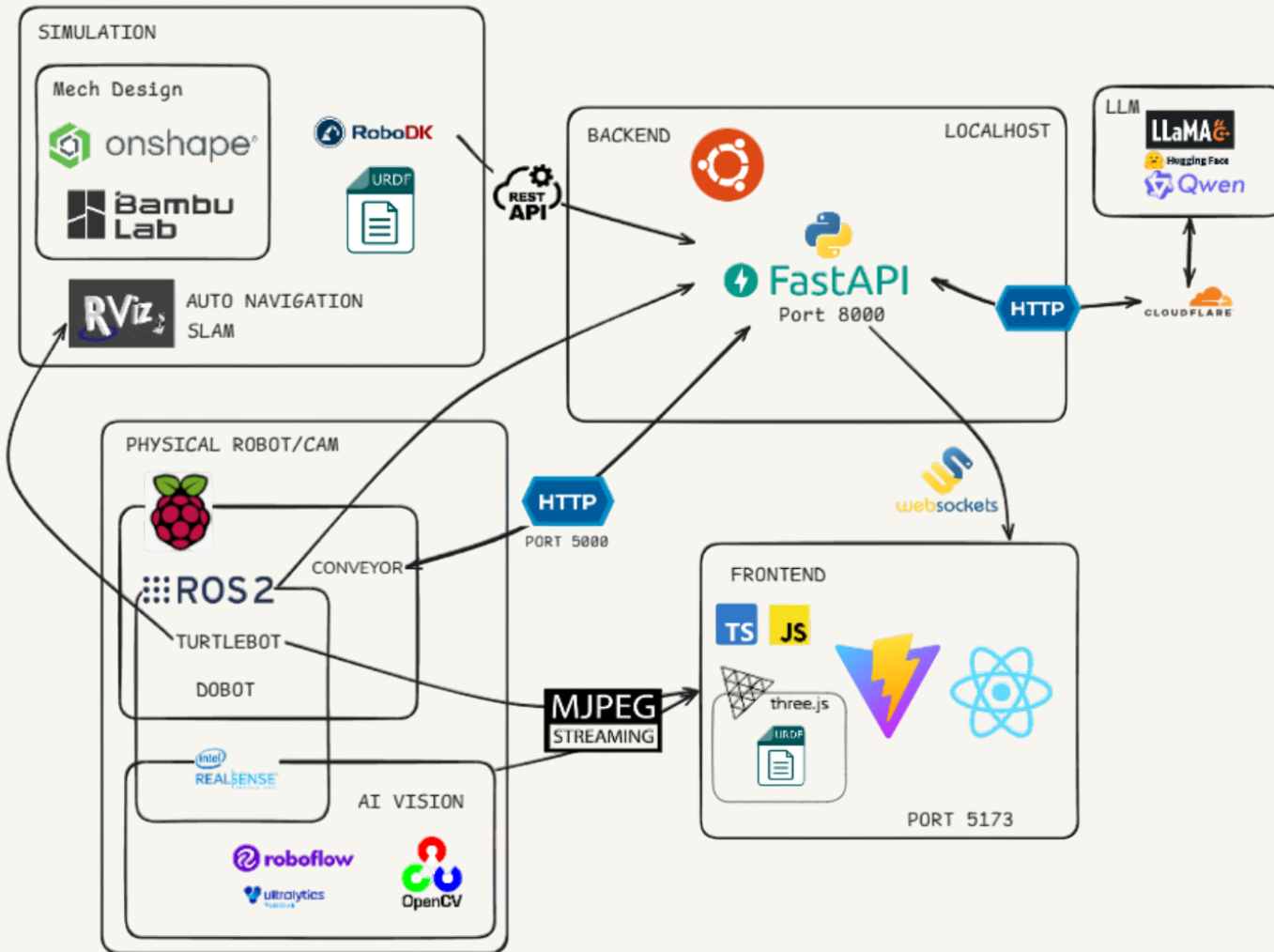
카메라의 검사 결과로 컨베이어를 제어하고 Dobot이 대상을 집어 분류합니다. RealOps에서 검사·작업 상태와 장비를 함께 관리합니다.

직접 맡은 일

React 관제와 FastAPI·WebSocket 서버, ROS 2 연결, 장비 adapter와 집기 시퀀스를 구현했습니다. 서버에서 검출 이벤트를 장비 명령으로 연결했습니다.

AQIS · 장비 사용 전 통합 경로 확보

초기 일정의 제약을 Mock/Real adapter와 공통 관제 인터페이스로 다뤘습니다.



제약: 마지막 3일

초기 계획에서 실물 장비 사용은 마지막 3일로 제한되었습니다. 앞선 기간에 관제·API·WebSocket 흐름을 먼저 연결하는 전략을 정했습니다.

선택: 연결부 분리

Mock 응답으로 개발한 관제 아래에 실제 장비 adapter를 연결했습니다. 서버의 공통 API와 상태 이벤트를 유지하면서 장비별 연결을 분리했습니다.

통합 범위

컨베이어·Dobot·검사 관제를 연결했습니다. 그림의 TurtleBot 자동 임무는 확장 설계이며, 구현은 SLAM·카메라·상태 관제까지입니다.

AQIS · 좌표 갱신과 집기 제어

이동 중의 최초 좌표를 계속 쓰지 않고, 정지 요청 뒤 후속 검출로 집기 위치를 갱신합니다.

1

검출 수신

대상·작업 상태 확인

2

> 정지 요청·대기

설정 시간 대기

3

> 좌표 재취득

후속 검출·깊이 좌표 우선

4

> 집기·분류·재개

스크립트 정상 종료 시 재개

검출과 실행의 중복 분리

같은 대상이 여러 프레임에 나타나므로 객체 ID와 영역 겹침·중심 거리로 중복 집계를 억제합니다. 집기 작업 중에는 명령의 중복 실행도 막습니다.

상태와 시각에 따른 입력 선택

관제가 정지되면 검출에 따른 통계 갱신과 집기를 시작하지 않습니다. timestamp가 있는 검출은 준비 시점 이전이거나 허용 연령을 넘으면 제외합니다.

실물 완료의 확인 조건

설정 시간 대기와 스크립트 종료 코드는 실제 정지·파지 성공의 센서 확인이 아닙니다. 자동 흐름은 정지 명령의 실패 응답으로 다음 단계를 차단하지 않습니다.

누락 입력과 검증 범위

timestamp가 없으면 신선도 검사를 통과하고, 깊이가 없으면 고정 좌표를 사용합니다. 좌표 변환 기대값 불일치와 실물 반복 성능·통신 복구는 남은 검증 항목입니다.

중복·정지 상태·시각 조건: 원본 테스트와 후속 Mock 재현으로 확인. 실물 성공률·사이클 시간의 측정 결과는 없습니다.

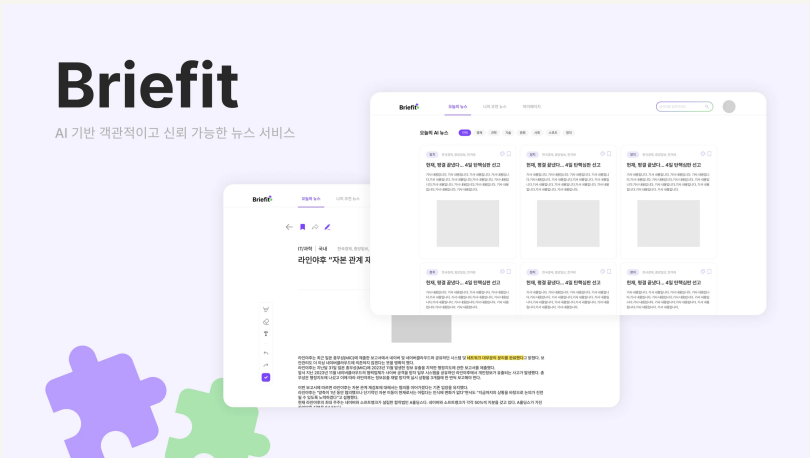
Briefit · 뉴스 요약과 출력 품질

여러 매체의 뉴스를 모아 요약을 제공하는 서비스 / 2025년 본인 AI 구현

기간
2025.05 - 09

팀·역할
6인 팀 / AI 2인
본인: AI 담당

사용 기술
Python / Crawl4AI
BeautifulSoup
Transformers / KoBART



서비스 UI · 팀 결과

직접 구현한 범위

기사 본문 정제와 배치 간 URL 중복 방지, KoBART 학습·생성·평가 스크립트를 구현했습니다. 긴 기사의 부분 요약·재요약과 생성문 후처리를 맡았습니다.

구현을 나눈 기준

수집 입력, 모델 생성, 출력 후처리를 분리해 기사 중복과 요약 누락·반복을 각각 점검할 수 있게 구성했습니다.

프로젝트 수상

2025 IT대학 소프트웨어 공모전 금상

2025.08.18 / Briefit

[상장 보기](#)

제15회 숭실 캡스톤디자인 경진대회 장려상

2025.10.01 / Briefit

[상장 보기](#)

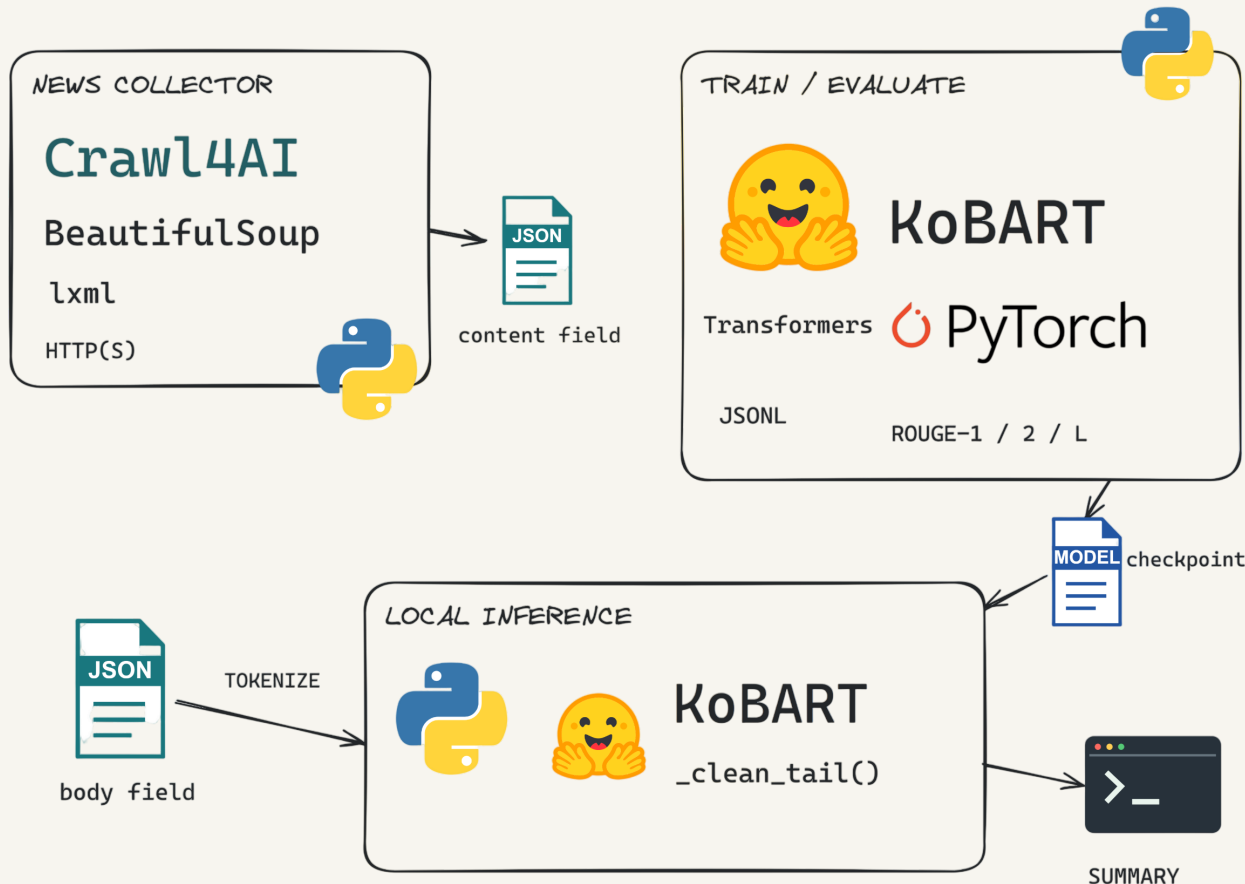
2025 IT 프로젝트 프로리그 장려상

2025.11.22 / Briefit

[상장 보기](#)

Briefit · 긴 기사와 요약 손실

입력 중복, 모델의 길이 제한, 후처리의 정보 손실을 서로 다른 문제로 다뤘습니다.



본문 정제와 중복 방지

본문에서 스크립트·스타일을 제거하고 댓글 URL을 제외합니다. 같은 실행의 여러 배치가 URL 집합을 공유해 이미 수집한 기사의 재저장을 막습니다.

부분 요약 뒤 재요약

긴 입력은 문단을 묶어 부분 요약한 뒤 다시 요약합니다. 긴 문단이나 재요약 입력은 길이 제한으로 잘릴 수 있어 전체 문맥 보존을 보장하지는 않습니다.

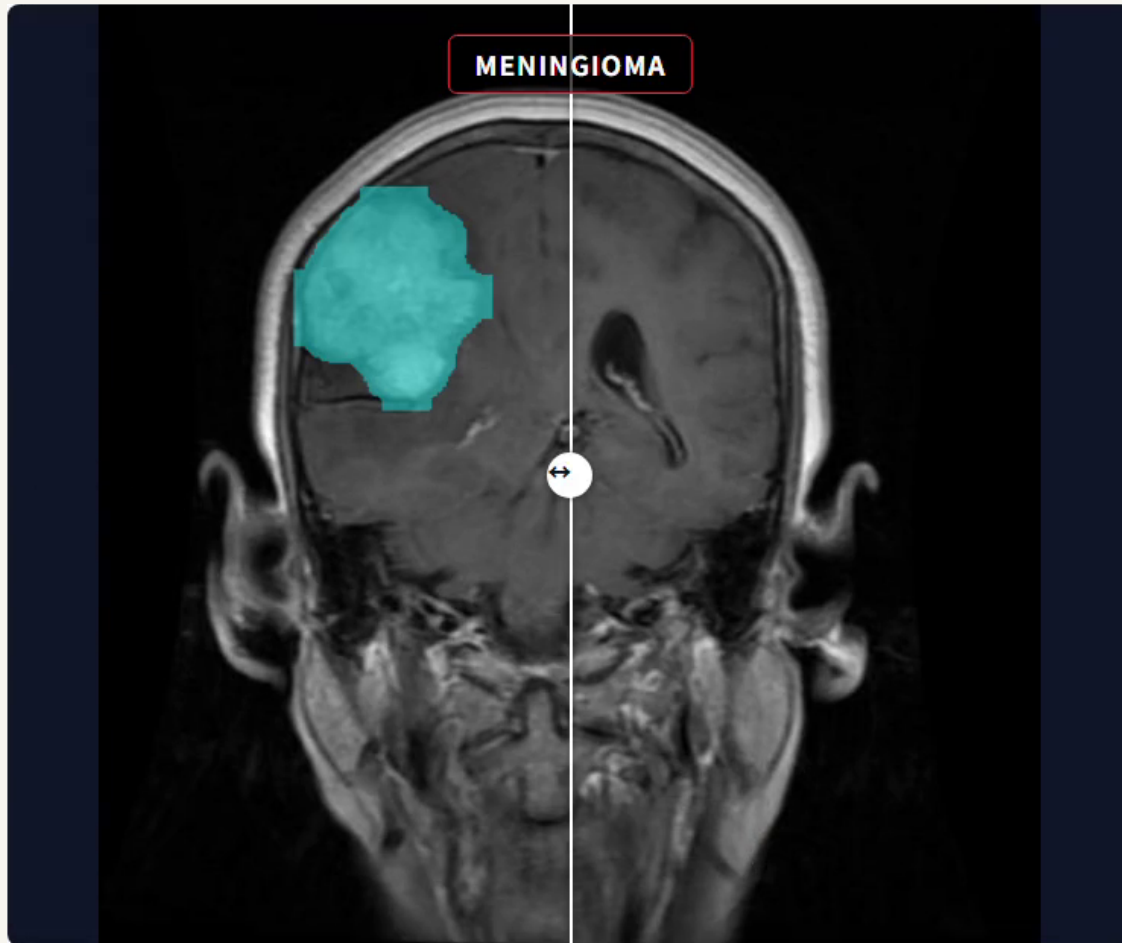
후처리와 평가의 분리

반복·짧은 끝문장을 제거하는 규칙은 정상 문장도 지울 수 있습니다. ROUGE는 후처리 전 생성문을 평가하므로 반복 감소와 정보 손실은 별도 비교가 필요합니다.

검증 범위: 생성·평가 코드 확인. ROUGE 점수 로그와 후처리 전후 품질 비교 결과는 확인되지 않았습니다.

Brain MRI · 종양 분류와 영역 분할

개인 프로젝트 / 2026.02 - 04 / Python · PyTorch · YOLO11 · OpenCV



분류·분할 통합 추론 · 원본 데모

프로젝트 개요

MRI의 종양 유형을 분류하고 영역을 분할하는 연구·학습용 프로젝트입니다. 분류 범주와 분할 위치를 같은 이미지에서 비교할 수 있도록 구성했습니다.

직접 구현한 범위

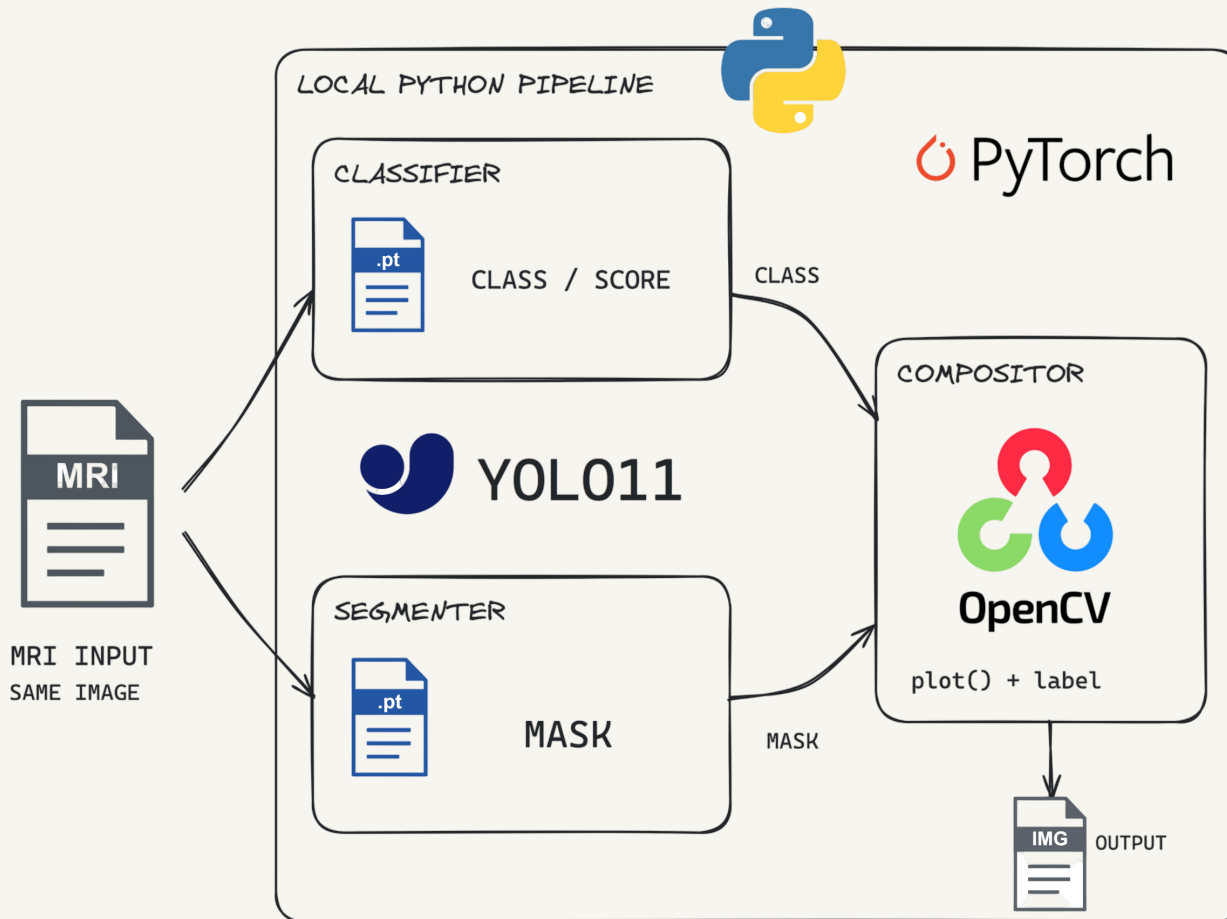
BRISC 이미지·마스크를 학습 형식으로 변환하고, 두 YOLO11 모델의 학습과 통합 추론 코드를 작성했습니다. 데이터 전처리부터 예측 시각화까지 연결했습니다.

학습 입력: 분류용 이미지 폴더 / 분할용 polygon label

출력: 분류 범주와 분할 영역의 통합 시각화

Brain MRI · 분류·분할 모델 구조

분류와 분할은 독립적으로 추론하고, 라벨 변환과 평가 데이터의 조건을 별도로 관리합니다.



라벨 정제와 영역 손실

마스크의 작은 영역을 제거하고 외부 윤곽을 polygon label로 변환합니다. 잡음을 줄이는 규칙이 작은 병변이나 내부 구멍도 지울 수 있어 원본 마스크와 대조가 필요합니다.

분류가 분할을 차단하지 않도록

같은 MRI를 두 모델에 각각 입력합니다. 비종양으로 분류해도 분할 추론을 실행하며, 두 결과를 함께 표시합니다. 결과 불일치를 자동으로 보정하는 구조는 아닙니다.

평가 데이터의 독립성

학습 코드는 test를 검증용 val에 연결합니다. 학습·검증에 쓰지 않은 별도 평가가 필요하며, BRISC에는 환자 식별자가 없어 환자 단위 독립성도 확인할 수 없습니다.

연구용 데이터의 비종양 범주에도 병변이 포함될 수 있습니다. 임상 진단 검증과 독립 성능 평가는 완료하지 않았습니다.

Prompt Generator · 프로젝트 설계 도우미

아이디어를 영역별 질문으로 구체화하고 결과를 하나의 문서로 모으는 도구

기간

2026

팀·역할

개인 프로젝트

대화 서버·상태 관리

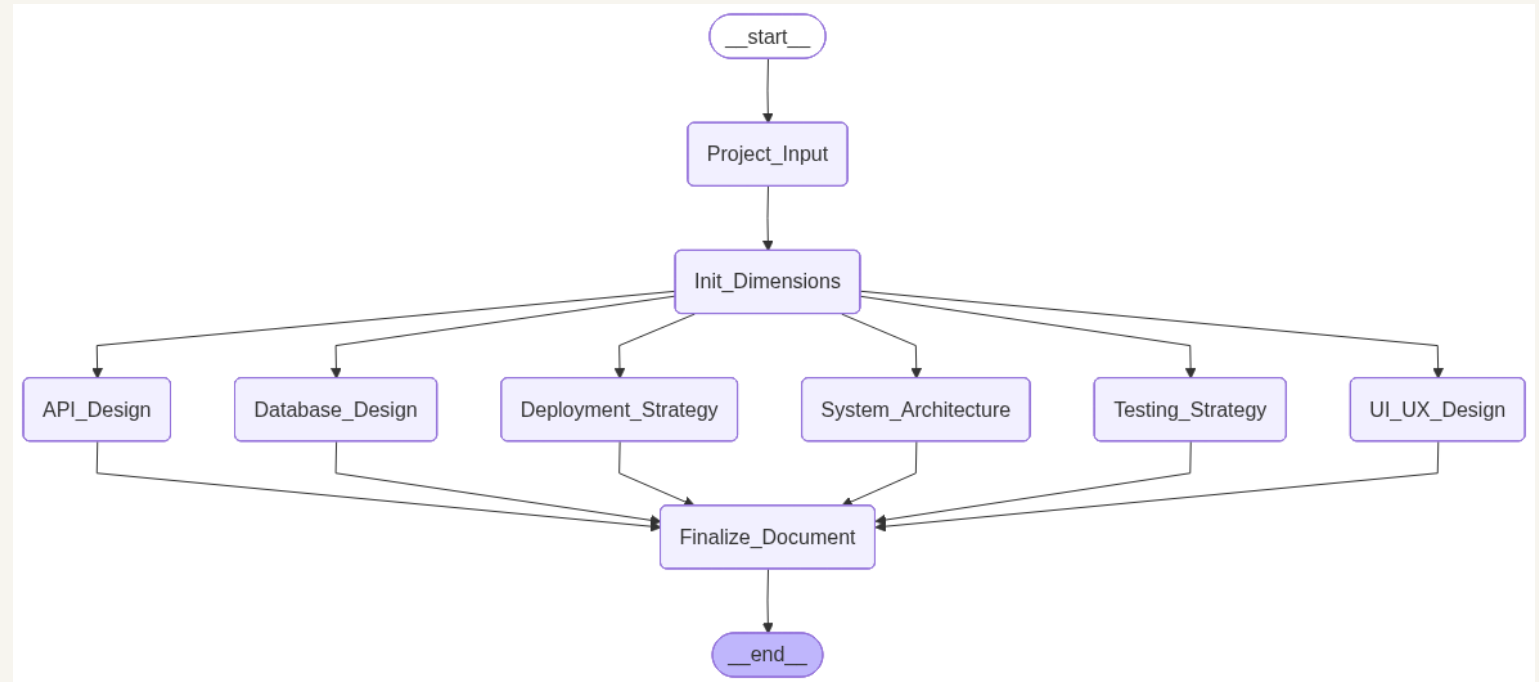
웹 UI

사용 기술

Python / FastAPI

WebSocket / LangChain

Solar Pro



초기 설계 흐름 · 원본

프로젝트 개요

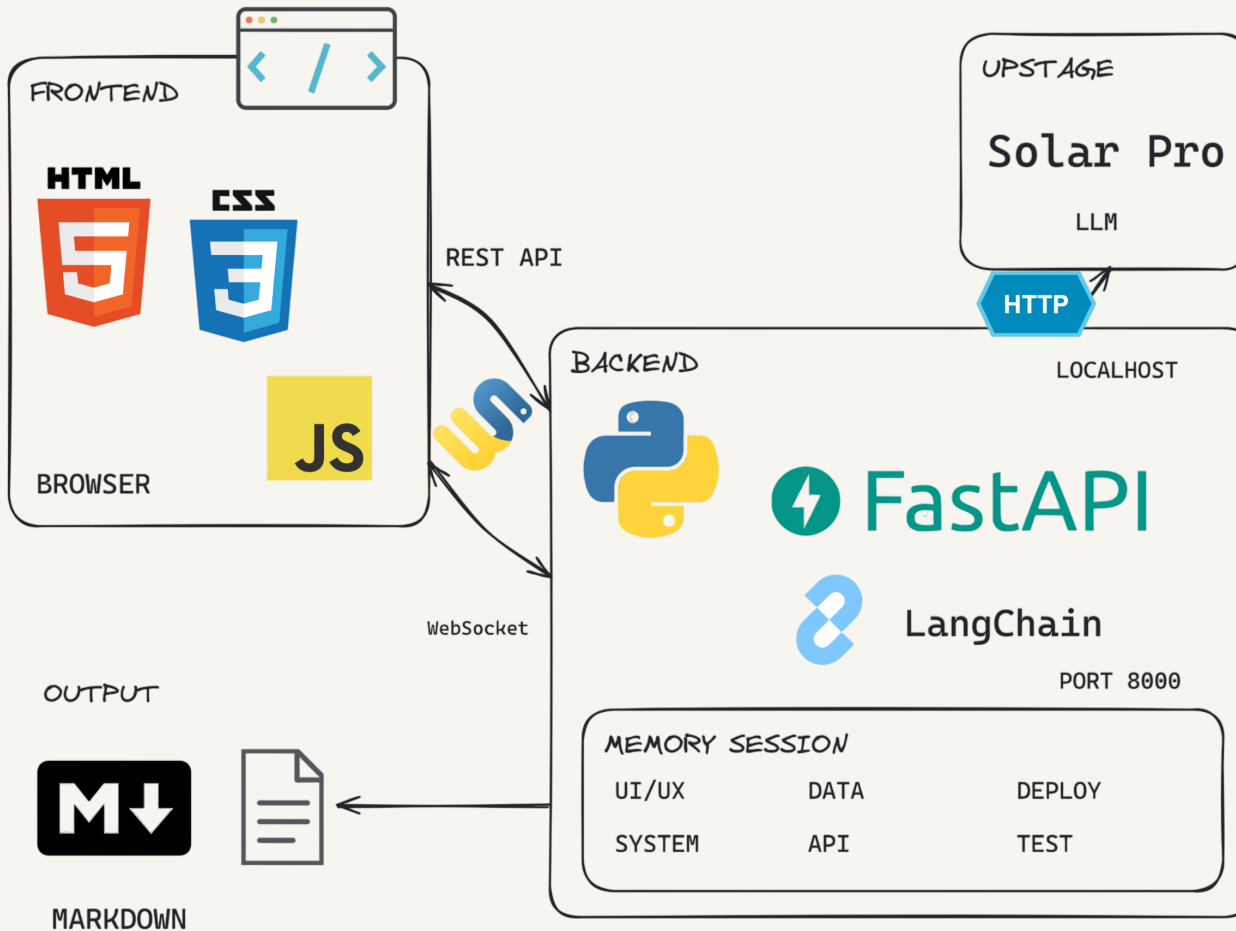
프로젝트 아이디어를 입력하면 화면·API·데이터 등 영역별 질문으로 요구사항을 구체화합니다. 각 영역의 결과를 하나의 설계 문서로 모읍니다.

직접 맡은 일

대화 서버와 웹 UI를 연결하고, 질문·답변에서 결과 문서 생성까지 진행 상태를 관리했습니다. 완료된 영역을 추가 대화로 수정하는 흐름도 구현했습니다.

Prompt Generator · 영역별 설계 대화

대화 문맥, 완료 조건과 호출 오류를 각각의 상태 전환으로 다룹니다.



영역별 문맥 분리

여섯 영역의 첫 질문을 병렬로 시작하고, 각 영역의 대화 이력·진행 상태·결과를 따로 유지합니다. 한 영역의 답변을 다른 영역의 대화에 섞지 않는 구조입니다.

질문과 완료 결과의 구분

처리 라운드 조건과 결과 생성 태그를 함께 확인해 완료로 전환합니다. 조건이 맞지 않으면 질문을 이어가며, 이 조건이 생성 품질을 판정하는 것은 아닙니다.

호출 오류 시 상태 처리

처리 대상 LLM 오류가 나면 라운드를 되돌리고 해당 영역을 대기 상태로 바꿉니다. 입력·응답은 호출 성공 뒤 이력에 추가하며 자동 재시도는 하지 않습니다.

세션은 연결 종료 시 삭제되며 재접속 복구는 없습니다. 요구사항 반영률·생성 품질의 평가 결과는 확인되지 않았습니다.

Alkkagi.io · 실시간 알까기

여러 플레이어가 같은 보드에서 돌을 밀어내며 대전하는 웹 게임

기간

2026.03 - 04

팀·역할

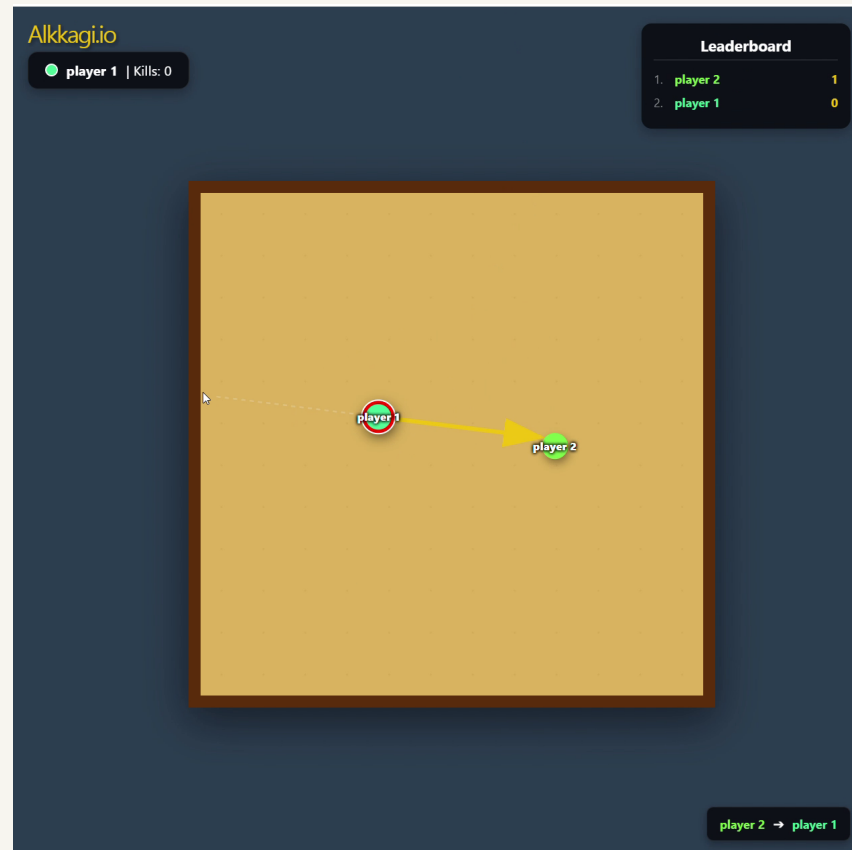
개인 프로젝트

웹 UI·서버·물리 계산

사용 기술

React / TypeScript

Node.js / Socket.IO



실제 플레이

프로젝트 개요

같은 보드에 접속한 플레이어가 돌을 밀어내며 대전하는 웹 게임입니다. 돌의 움직임과 충돌 결과를 실시간으로 공유합니다.

직접 맡은 일

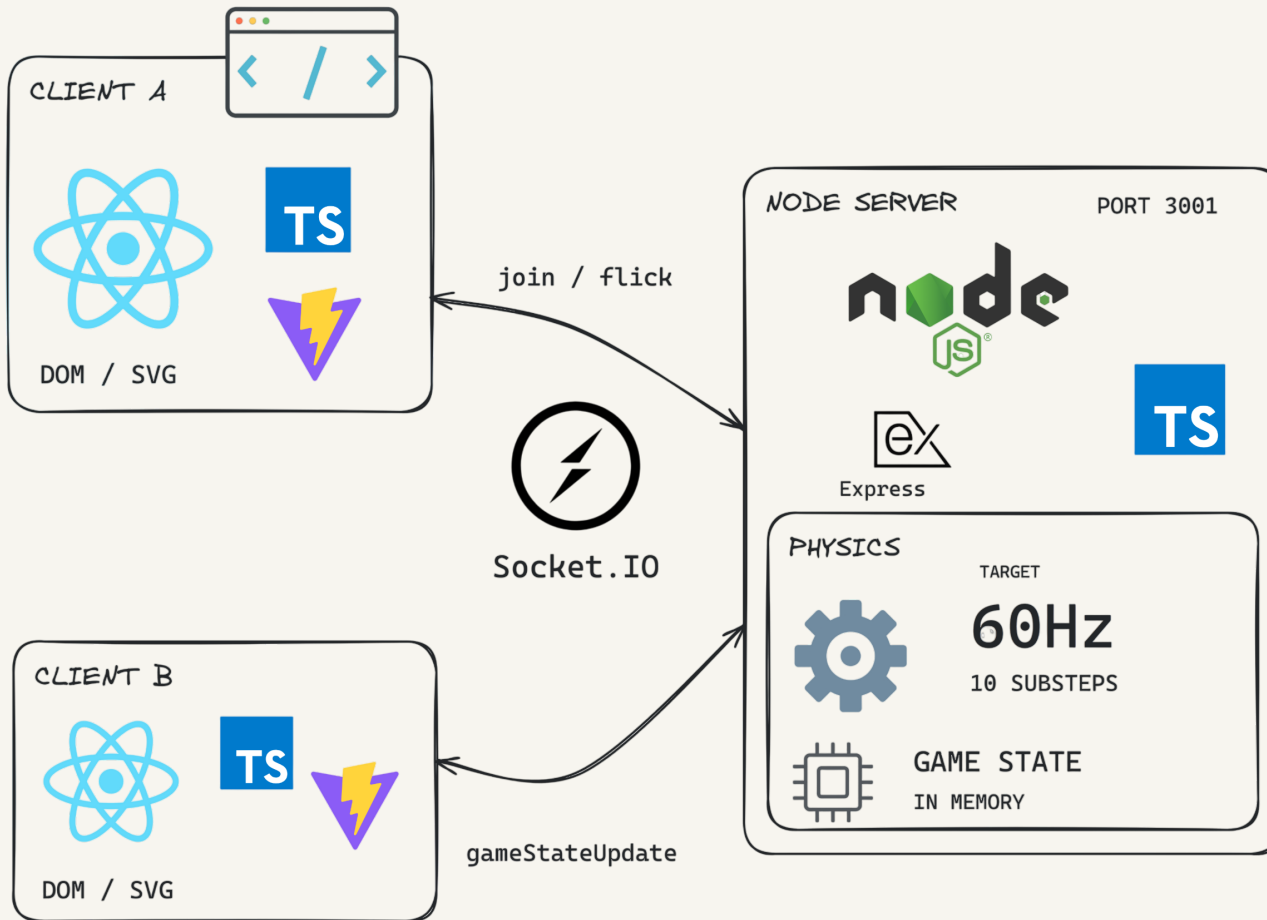
React 조준 UI와 Socket.IO 통신, 서버의 게임 상태·물리 계산을 구현했습니다. 충돌·마찰·검침 보정은 TypeScript로 작성했습니다.

게임 규칙의 실행 위치

클라이언트는 발사 입력을 보내고 서버가 기준 상태를 계산합니다. 입력 제한과 물리 규칙을 모든 플레이어에게 같은 기준으로 적용합니다.

Alkkagi.io · 서버 물리와 상태 동기화

입력 제한, 겹침 보정과 충돌 반응을 분리하고 계산이 끝난 상태를 전송합니다.



겹침과 충격량의 분리

겹친 거리는 두 돌에 절반씩 나눠 위치를 보정합니다.
이미 서로 멀어지면 충격량을 더하지 않고, 접근 중인 충돌에는 질량을 반영해 속도를 갱신합니다.

계산 간격과 마찰

한 번의 갱신을 소단계로 나눠 이동·충돌을 계산합니다.
마찰 계수에도 소단계 비율을 지수로 적용해 계산을 나눌 때 감속이 과도하게 누적되지 않도록 했습니다.

서버에서 입력 검증

발사 간격과 속도를 서버에서 제한합니다. 기준 상태는 서버 메모리에 두고 연결 종료 시 플레이어를 정리합니다.
재접속·서버 재시작 후 복구는 구현하지 않았습니다.

구조도의 60Hz는 갱신 설정입니다. 동시접속 부하·네트워크 지연·지속 프레임률의 측정 결과는 없습니다.

연락처

Se Min Kong

공세민 · 소프트웨어 개발자

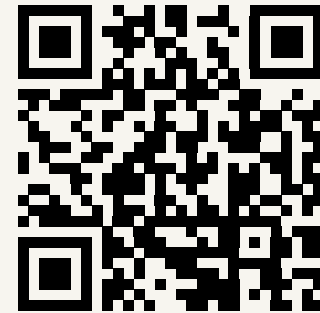
이메일

semin1224@gmail.com

GitHub

github.com/SeMinKong

웹 포트폴리오



seminkong.github.io/SeMinKong_Web/

프로젝트 영상과 상세 설명

이력서와 상장 전시